

Linebuffer

```
module lineBuffer(
input i_clk,
input i_rst,
input [7:0] i_data,
input i_data_valid,
output [23:0] o_data,
input i_rd_data
);

reg [7:0] line [511:0]; //line buffer
reg [8:0] wrPntr;
reg [8:0] rdPntr;

always @(posedge i_clk)
begin
    if(i_data_valid)
        line[wrPntr] <= i_data;
end

always @(posedge i_clk)
begin
    if(i_rst)
        wrPntr <= 'd0;
    else if(i_data_valid)
        wrPntr <= wrPntr + 'd1;
end

assign o_data = {line[rdPntr],line[rdPntr+1],line[rdPntr+2]};

always @(posedge i_clk)
begin
    if(i_rst)
        rdPntr <= 'd0;
    else if(i_rd_data)
        rdPntr <= rdPntr + 'd1;
end

endmodule
```

Conv.v

```
module conv(
input    i_clk,
input [71:0] i_pixel_data,
input    i_pixel_data_valid,
output reg [7:0] o_convolved_data,
output reg o_convolved_data_valid
);

integer i;
reg [7:0] kernel [8:0];
reg [15:0] multData[8:0];
reg [15:0] sumDataInt;
reg [15:0] sumData;
reg multDataValid;
reg sumDataValid;
reg convolved_data_valid;

initial
begin
    for(i=0;i<9;i=i+1)
        begin
            kernel[i] = 1;
        end
    end

always @(posedge i_clk)
begin
    for(i=0;i<9;i=i+1)
        begin
            multData[i] <= kernel[i]*i_pixel_data[i*8+:8];
        end
    multDataValid <= i_pixel_data_valid;
end

always @(*)
begin
    sumDataInt = 0;
    for(i=0;i<9;i=i+1)
        begin
            sumDataInt = sumDataInt + multData[i];
        end
end
```

```

always @(posedge i_clk)
begin
    sumData <= sumDataInt;
    sumDataValid <= multDataValid;
end

always @(posedge i_clk)
begin
    o_convolved_data <= sumData/9;
    o_convolved_data_valid <= sumDataValid;
end

endmodule

```

image control.v

```

\
module imageControl(
input          i_clk,
input          i_rst,
input [7:0]    i_pixel_data,
input          i_pixel_data_valid,
output reg [71:0] o_pixel_data,
output         o_pixel_data_valid,
output reg     o_intr
);

reg [8:0] pixelCounter;
reg [1:0] currentWrLineBuffer;
reg [3:0] lineBuffDataValid;
reg [3:0] lineBuffRdData;
reg [1:0] currentRdLineBuffer;
wire [23:0] lb0data;
wire [23:0] lb1data;
wire [23:0] lb2data;
wire [23:0] lb3data;
reg [8:0] rdCounter;
reg rd_line_buffer;
reg [11:0] totalPixelCounter;
reg rdState;

localparam IDLE = 'b0,
            RD_BUFFER = 'b1;

assign o_pixel_data_valid = rd_line_buffer;

```

```

always @(posedge i_clk)
begin
    if(i_rst)
        totalPixelCounter <= 0;
    else
        begin
            if(i_pixel_data_valid & !rd_line_buffer)
                totalPixelCounter <= totalPixelCounter + 1;
            else if(!i_pixel_data_valid & rd_line_buffer)
                totalPixelCounter <= totalPixelCounter - 1;
        end
    end
end

```

```

always @(posedge i_clk)
begin
    if(i_rst)
        begin
            rdState <= IDLE;
            rd_line_buffer <= 1'b0;
            o_intr <= 1'b0;
        end
    else
        begin
            case(rdState)
                IDLE:begin
                    o_intr <= 1'b0;
                    if(totalPixelCounter >= 1536)
                        begin
                            rd_line_buffer <= 1'b1;
                            rdState <= RD_BUFFER;
                        end
                end
                RD_BUFFER:begin
                    if(rdCounter == 511)
                        begin
                            rdState <= IDLE;
                            rd_line_buffer <= 1'b0;
                            o_intr <= 1'b1;
                        end
                end
            endcase
        end
    end
end

```

```

always @(posedge i_clk)
begin

```

```

    if(i_rst)
        pixelCounter <= 0;
    else
        begin
            if(i_pixel_data_valid)
                pixelCounter <= pixelCounter + 1;
            end
        end
    end
end

```

```

always @(posedge i_clk)
begin
    if(i_rst)
        currentWrLineBuffer <= 0;
    else
        begin
            if(pixelCounter == 511 & i_pixel_data_valid)
                currentWrLineBuffer <= currentWrLineBuffer+1;
            end
        end
    end
end

```

```

always @(*)
begin
    lineBuffDataValid = 4'h0;
    lineBuffDataValid[currentWrLineBuffer] = i_pixel_data_valid;
end

```

```

always @(posedge i_clk)
begin
    if(i_rst)
        rdCounter <= 0;
    else
        begin
            if(rd_line_buffer)
                rdCounter <= rdCounter + 1;
            end
        end
    end
end

```

```

always @(posedge i_clk)
begin
    if(i_rst)
        begin
            currentRdLineBuffer <= 0;
        end
    else

```

```

begin
    if(rdCounter == 511 & rd_line_buffer)
        currentRdLineBuffer <= currentRdLineBuffer + 1;
    end
end

```

```

always @(*)
begin
    case(currentRdLineBuffer)
        0:begin
            o_pixel_data = {lb2data,lb1data,lb0data};
        end
        1:begin
            o_pixel_data = {lb3data,lb2data,lb1data};
        end
        2:begin
            o_pixel_data = {lb0data,lb3data,lb2data};
        end
        3:begin
            o_pixel_data = {lb1data,lb0data,lb3data};
        end
    endcase
end

```

```

always @(*)
begin
    case(currentRdLineBuffer)
        0:begin
            lineBuffRdData[0] = rd_line_buffer;
            lineBuffRdData[1] = rd_line_buffer;
            lineBuffRdData[2] = rd_line_buffer;
            lineBuffRdData[3] = 1'b0;
        end
        1:begin
            lineBuffRdData[0] = 1'b0;
            lineBuffRdData[1] = rd_line_buffer;
            lineBuffRdData[2] = rd_line_buffer;
            lineBuffRdData[3] = rd_line_buffer;
        end
        2:begin
            lineBuffRdData[0] = rd_line_buffer;
            lineBuffRdData[1] = 1'b0;
            lineBuffRdData[2] = rd_line_buffer;
            lineBuffRdData[3] = rd_line_buffer;
        end
    end
end

```

```

        3:begin
            lineBuffRdData[0] = rd_line_buffer;
            lineBuffRdData[1] = rd_line_buffer;
            lineBuffRdData[2] = 1'b0;
            lineBuffRdData[3] = rd_line_buffer;
        end
    endcase
end

```

```

lineBuffer lB0(
    .i_clk(i_clk),
    .i_rst(i_rst),
    .i_data(i_pixel_data),
    .i_data_valid(lineBuffDataValid[0]),
    .o_data(lb0data),
    .i_rd_data(lineBuffRdData[0])
);

```

```

lineBuffer lB1(
    .i_clk(i_clk),
    .i_rst(i_rst),
    .i_data(i_pixel_data),
    .i_data_valid(lineBuffDataValid[1]),
    .o_data(lb1data),
    .i_rd_data(lineBuffRdData[1])
);

```

```

lineBuffer lB2(
    .i_clk(i_clk),
    .i_rst(i_rst),
    .i_data(i_pixel_data),
    .i_data_valid(lineBuffDataValid[2]),
    .o_data(lb2data),
    .i_rd_data(lineBuffRdData[2])
);

```

```

lineBuffer lB3(
    .i_clk(i_clk),
    .i_rst(i_rst),
    .i_data(i_pixel_data),
    .i_data_valid(lineBuffDataValid[3]),
    .o_data(lb3data),
    .i_rd_data(lineBuffRdData[3])
);

```

```

endmodule

```

imageProcessTop.v

```
module imageProcessTop1(
input  axi_clk,
input  axi_reset_n,
//slave interface
input  i_data_valid,
input [7:0] i_data,
output o_data_ready,
//master interface
output o_data_valid,
output [7:0] o_data,
input  i_data_ready,
//interrupt
output o_intr

);

wire [71:0] pixel_data;
wire pixel_data_valid;
wire axis_prog_full;
wire [7:0] convolved_data;
wire convolved_data_valid;

assign o_data_ready = !axis_prog_full;

imageControl IC(
.i_clk(axi_clk),
.i_rst(!axi_reset_n),
.i_pixel_data(i_data),
.i_pixel_data_valid(i_data_valid),
.o_pixel_data(pixel_data),
.o_pixel_data_valid(pixel_data_valid),
.o_intr(o_intr)
);

conv conv(
.i_clk(axi_clk),
.i_pixel_data(pixel_data),
.i_pixel_data_valid(pixel_data_valid),
.o_convolved_data(convolved_data),
.o_convolved_data_valid(convolved_data_valid)
);

outputBuffer OB (
.wr_rst_busy(), // output wire wr_rst_busy
```



```

.rd_rst_busy(),    // output wire rd_rst_busy
.s_aclk(axi_clk),    // input wire s_aclk
.s_aresetn(axi_reset_n),    // input wire s_aresetn
.s_axis_tvalid(convolved_data_valid),    // input wire s_axis_tvalid
.s_axis_tready(),    // output wire s_axis_tready
.s_axis_tdata(convolved_data),    // input wire [7 : 0] s_axis_tdata
.m_axis_tvalid(o_data_valid),    // output wire m_axis_tvalid
.m_axis_tready(i_data_ready),    // input wire m_axis_tready
.m_axis_tdata(o_data),    // output wire [7 : 0] m_axis_tdata
.axis_prog_full(axis_prog_full) // output wire axis_prog_full
);

```

endmodule

rowinheritance

```

module conv(//row
    input    i_clk,
    input [71:0] i_pixel_data,
    input    i_pixel_data_valid,
    output reg [7:0] o_convolved_data,
    output reg    o_convolved_data_valid
);
integer i;
reg [7:0] kernel [8:0];
reg [15:0] multData[8:0];
reg [15:0] sumDataInt;
reg [15:0] sumData;
reg multDataValid;
reg sumDataValid;

reg [9:0] row_counter = 0;
reg [9:0] col_counter = 0;
reg [18:0] current_index;
reg [18:0] prev_index_1, prev_index_2, prev_index_3;
reg [7:0] output_memory [0:262143];
reg [7:0] inherited_pixel;
reg [7:0] final_pixel;
reg write_enable;

initial begin
    for (i = 0; i < 9; i = i + 1)

```

```

        kernel[i] = 1;
    end

    always @(posedge i_clk) begin
        for (i = 0; i < 9; i = i + 1)
            multData[i] <= kernel[i] * i_pixel_data[i*8+: 8];
        multDataValid <= i_pixel_data_valid;
    end

    always @(*) begin
        sumDataInt = 0;
        for (i = 0; i < 9; i = i + 1)
            sumDataInt = sumDataInt + multData[i];
    end

    always @(posedge i_clk) begin
        sumData <= sumDataInt;
        sumDataValid <= multDataValid;
    end

    always @(posedge i_clk) begin
        if (i_pixel_data_valid) begin
            if (col_counter == 511) begin
                col_counter <= 0;
                if (row_counter == 511)
                    row_counter <= 0;
                else
                    row_counter <= row_counter + 1;
            end else begin
                col_counter <= col_counter + 1;
            end
        end
    end

    always @(posedge i_clk) begin
        if (sumDataValid) begin

            current_index = row_counter * 512 + col_counter;
            prev_index_1 = (row_counter > 0) ? ((row_counter - 1) * 512 + col_counter) : 0;
            prev_index_2 = (row_counter > 1) ? ((row_counter - 2) * 512 + col_counter) : 0;
            prev_index_3 = (row_counter > 2) ? ((row_counter - 3) * 512 + col_counter) : 0;

            inherited_pixel = sumData / 9;

            if ((row_counter % 4 == 0) && row_counter >= 3)
                final_pixel = output_memory[prev_index_3];
            else if ((row_counter % 4 == 2) && row_counter >= 1)

```

```

        final_pixel = output_memory[prev_index_1];
    else if ((row_counter % 4 == 3) && row_counter >= 2)
        final_pixel = output_memory[prev_index_2];
    else
        final_pixel = inherited_pixel;

    o_convolved_data <= final_pixel;
    o_convolved_data_valid <= 1;

    output_memory[current_index] <= final_pixel;
end else begin
    o_convolved_data_valid <= 0;
end
end
endmodule

```

column inheritance

```

module conv(column
    input    i_clk,
    input [71:0] i_pixel_data,
    input    i_pixel_data_valid,
    output reg [7:0] o_convolved_data,
    output reg    o_convolved_data_valid
);

integer i;
reg [7:0] kernel [8:0];
reg [15:0] multData[8:0];
reg [15:0] sumDataInt;
reg [15:0] sumData;
reg multDataValid;
reg sumDataValid;

reg [9:0] row_counter = 0;
reg [9:0] col_counter = 0;
reg [18:0] current_index;
reg [18:0] prev_col_1, prev_col_2, prev_col_3;
reg [7:0] output_memory [0:262143];
reg [7:0] inherited_pixel;
reg [7:0] final_pixel;

initial begin
    for (i = 0; i < 9; i = i + 1)
        kernel[i] = 1;

```

end

```
// Multiply kernel with input pixels
always @(posedge i_clk) begin
    for (i = 0; i < 9; i = i + 1)
        multData[i] <= kernel[i] * i_pixel_data[i*8+: 8];
    multDataValid <= i_pixel_data_valid;
end
```

```
// Sum of all multiplied data
always @(*) begin
    sumDataInt = 0;
    for (i = 0; i < 9; i = i + 1)
        sumDataInt = sumDataInt + multData[i];
end
```

```
always @(posedge i_clk) begin
    sumData <= sumDataInt;
    sumDataValid <= multDataValid;
end
```

```
// Update row and column counters
always @(posedge i_clk) begin
    if (i_pixel_data_valid) begin
        if (col_counter == 511) begin
            col_counter <= 0;
            if (row_counter == 511)
                row_counter <= 0;
            else
                row_counter <= row_counter + 1;
        end else begin
            col_counter <= col_counter + 1;
        end
    end
end
```

```
// Column inheritance logic
always @(posedge i_clk) begin
    if (sumDataValid) begin
        current_index = row_counter * 512 + col_counter;
        prev_col_1 = (col_counter > 0) ? (row_counter * 512 + (col_counter - 1)) : 0;
        prev_col_2 = (col_counter > 1) ? (row_counter * 512 + (col_counter - 2)) : 0;
        prev_col_3 = (col_counter > 2) ? (row_counter * 512 + (col_counter - 3)) : 0;

        inherited_pixel = sumData / 9;
    end
end
```

```

// Inherit from previous columns based on column_counter % 4
if ((col_counter % 4 == 0) && col_counter >= 3)
    final_pixel = output_memory[prev_col_3];
else if ((col_counter % 4 == 2) && col_counter >= 1)
    final_pixel = output_memory[prev_col_1];
else if ((col_counter % 4 == 3) && col_counter >= 2)
    final_pixel = output_memory[prev_col_2];
else
    final_pixel = inherited_pixel;

o_convolved_data <= final_pixel;
o_convolved_data_valid <= 1;

output_memory[current_index] <= final_pixel;
end else begin
    o_convolved_data_valid <= 0;
end
end endmodule

```

```

// Adaptive Threshold Convolution
module conv_adaptive_threshold (
    input    i_clk,
    input [71:0] i_pixel_data,
    input    i_pixel_data_valid,
    output reg [7:0] o_convolved_data,
    output reg    o_convolved_data_valid
);

```

```

// Internal states
integer i;
reg [7:0] kernel [0:8];
reg [15:0] multData [0:8];
reg [15:0] sumData;
reg [7:0] conv_pixel;

```

```

// Image context
reg [9:0] row_counter = 0;
reg [9:0] col_counter = 0;
reg [18:0] current_index;

```

```

// Memory to hold intermediate and final output
reg [7:0] output_memory [0:262143]; // For 512x512 image
reg [7:0] previous_row_pixels [0:511];

```

```

// Adaptive logic
reg [7:0] pixel_ij, pixel_ilj, pixel_ijl;

```

```
reg [7:0] final_pixel;  
parameter THRESHOLD = 20; // Fixed threshold as approximation of adaptthresh
```

```
// Kernel initialization (box blur)
```

```
initial begin
```

```
    for (i = 0; i < 9; i = i + 1)
```

```
        kernel[i] = 1;
```

```
end
```

```
// Step 1: Multiply convolution kernel
```

```
always @(posedge i_clk) begin
```

```
    for (i = 0; i < 9; i = i + 1)
```

```
        multData[i] <= kernel[i] * i_pixel_data[i*8+: 8];
```

```
end
```

```
// Step 2: Sum convolution result
```

```
always @(*) begin
```

```
    sumData = 0;
```

```
    for (i = 0; i < 9; i = i + 1)
```

```
        sumData = sumData + multData[i];
```

```
    conv_pixel = sumData / 9; // Box blur average
```

```
end
```

```
// Step 3: Row/column count update
```

```
always @(posedge i_clk) begin
```

```
    if (i_pixel_data_valid) begin
```

```
        if (col_counter == 511) begin
```

```
            col_counter <= 0;
```

```
            if (row_counter == 511)
```

```
                row_counter <= 0;
```

```
            else
```

```
                row_counter <= row_counter + 1;
```

```
        end else begin
```

```
            col_counter <= col_counter + 1;
```

```
        end
```

```
    end
```

```
end
```

```
// Step 4: Adaptive inheritance logic
```

```
always @(posedge i_clk) begin
```

```
    if (i_pixel_data_valid) begin
```

```
        // Compute memory index
```

```
        current_index = row_counter * 512 + col_counter;
```

```
        // Grab neighbors
```

```
        pixel_ij = conv_pixel;
```

```

pixel_ij = (row_counter > 0) ? previous_row_pixels[col_counter] : conv_pixel;
pixel_ij1 = (col_counter > 0) ? output_memory[current_index - 1] : conv_pixel;

// Adaptive threshold condition
if (((pixel_ij - pixel_ij1) % (col_counter + 1)) < THRESHOLD)
    final_pixel = pixel_ij1; // Inherit from previous column
else
    final_pixel = pixel_ij; // Use current convolved value

// Output
o_convolved_data <= final_pixel;
o_convolved_data_valid <= 1;

// Store results
output_memory[current_index] <= final_pixel;
previous_row_pixels[col_counter] <= conv_pixel; // Save for next row usage
end else begin
    o_convolved_data_valid <= 0;
end end endmodule

Cross inheritance
module conv (
    input        i_clk,
    input        i_rst,
    input  [71:0] i_pixel_data,
    input        i_pixel_data_valid,
    output reg [7:0] o_convolved_data,
    output reg    o_convolved_data_valid
);

integer i;

// 3x3 kernel weights (all ones for averaging)
reg [7:0] kernel[0:8];
initial begin
    for (i = 0; i < 9; i = i + 1)
        kernel[i] = 8'd1;
end

// Intermediate variables
reg [15:0] multData[0:8];
reg [15:0] sumDataInt;
reg [15:0] sumData;
reg    multDataValid, sumDataValid;

// Inheritance state
reg [7:0] row_buffer[0:511]; // Buffer to store previous row values (for top inheritance)

```

```

reg [7:0] left_pixel;      // Previous pixel value in current row (for left inheritance)

// Control counters
reg [9:0] row_counter = 0;
reg [9:0] col_counter = 0;

// Final inherited pixel logic
reg [7:0] conv_result;
reg [7:0] top_pixel;
reg [7:0] left_pixel_curr;

// Step 1: Multiply kernel with pixel data
always @(posedge i_clk) begin
    for (i = 0; i < 9; i = i + 1)
        multData[i] <= kernel[i] * i_pixel_data[i*8 +: 8];
    multDataValid <= i_pixel_data_valid;
end

// Step 2: Sum the multiplied results (combinational)
always @(*) begin
    sumDataInt = 0;
    for (i = 0; i < 9; i = i + 1)
        sumDataInt = sumDataInt + multData[i];
end

// Step 3: Store the final sum on clock edge
always @(posedge i_clk) begin
    sumData <= sumDataInt;
    sumDataValid <= multDataValid;
end

// Step 4: Update row and column counters
always @(posedge i_clk) begin
    if (i_pixel_data_valid) begin
        if (col_counter == 511) begin
            col_counter <= 0;
            if (row_counter == 511)
                row_counter <= 0;
        else
            row_counter <= row_counter + 1;
        end else begin
            col_counter <= col_counter + 1;
        end
    end
end
end

```



```

// Step 5: Final output with 75% crossing inheritance logic
always @(posedge i_clk) begin
    if (i_rst) begin
        o_convolved_data    <= 0;
        o_convolved_data_valid <= 0;
        left_pixel          <= 0;
    end else if (sumDataValid) begin
        conv_result    <= sumData / 9;
        top_pixel      <= row_buffer[col_counter];
        left_pixel_curr <= left_pixel;

        // 75% CROSSING INHERITANCE LOGIC
        if ((row_counter % 2 == 0) && (col_counter % 2 == 1))
            o_convolved_data <= top_pixel; // Inherit from top
        else if ((row_counter % 2 == 1) && (col_counter % 2 == 0))
            o_convolved_data <= left_pixel_curr; // Inherit from left
        else if ((row_counter % 2 == 0) && (col_counter % 2 == 0))
            o_convolved_data <= (top_pixel + left_pixel_curr) >> 1; // Average
        else
            o_convolved_data <= conv_result; // Use convolved result

        o_convolved_data_valid <= 1;

        // Update buffers
        row_buffer[col_counter] <= conv_result;
        left_pixel          <= conv_result;
    end else begin
        o_convolved_data_valid <= 0;
    end
end

endmodule

50%
module conv (
    input      i_clk,
    input      i_rst,
    input  [71:0] i_pixel_data,
    input      i_pixel_data_valid,
    output reg [7:0] o_convolved_data,
    output reg      o_convolved_data_valid
);

integer i;

reg [7:0] kernel[0:8];

```

```

initial begin
    for (i = 0; i < 9; i = i + 1)
        kernel[i] = 8'd1;
end

reg [15:0] multData[0:8];
reg [15:0] sumDataInt;
reg [15:0] sumData;
reg      multDataValid, sumDataValid;

reg [7:0] row_buffer[0:511];
reg [7:0] left_pixel;

reg [9:0] row_counter = 0;
reg [9:0] col_counter = 0;

reg [7:0] conv_result;
reg [7:0] top_pixel;
reg [7:0] left_pixel_curr;

always @(posedge i_clk) begin
    for (i = 0; i < 9; i = i + 1)
        multData[i] <= kernel[i] * i_pixel_data[i*8+: 8];
    multDataValid <= i_pixel_data_valid;
end

always @(*) begin
    sumDataInt = 0;
    for (i = 0; i < 9; i = i + 1)
        sumDataInt = sumDataInt + multData[i];
end

always @(posedge i_clk) begin
    sumData <= sumDataInt;
    sumDataValid <= multDataValid;
end

always @(posedge i_clk) begin
    if (i_pixel_data_valid) begin
        if (col_counter == 511) begin
            col_counter <= 0;
            if (row_counter == 511)
                row_counter <= 0;
            else
                row_counter <= row_counter + 1;
        end else begin

```

```

        col_counter <= col_counter + 1;
    end
end
end

always @(posedge i_clk) begin
    if (i_rst) begin
        o_convolved_data    <= 0;
        o_convolved_data_valid <= 0;
        left_pixel          <= 0;
    end else if (sumDataValid) begin
        conv_result    <= sumData / 9;
        top_pixel      <= row_buffer[col_counter];
        left_pixel_curr <= left_pixel;

        if (((row_counter + col_counter) % 2) == 0)
            o_convolved_data <= (top_pixel + left_pixel_curr) >> 1;
        else
            o_convolved_data <= conv_result;

        o_convolved_data_valid <= 1;
        row_buffer[col_counter] <= conv_result;
        left_pixel <= conv_result;
    end else begin
        o_convolved_data_valid <= 0;
    end
end
endmodule

```

25%

```

module conv (
    input      i_clk,
    input      i_rst,
    input  [71:0] i_pixel_data,
    input      i_pixel_data_valid,
    output reg [7:0] o_convolved_data,
    output reg      o_convolved_data_valid
);

```

```

    integer i;

```

```

    reg [7:0] kernel[0:8];
    initial begin
        for (i = 0; i < 9; i = i + 1)
            kernel[i] = 8'd1;
    end

```

```

end

reg [15:0] multData[0:8];
reg [15:0] sumDataInt;
reg [15:0] sumData;
reg      multDataValid, sumDataValid;

reg [7:0] row_buffer[0:511];
reg [7:0] left_pixel;

reg [9:0] row_counter = 0;
reg [9:0] col_counter = 0;

reg [7:0] conv_result;
reg [7:0] top_pixel;
reg [7:0] left_pixel_curr;

always @(posedge i_clk) begin
    for (i = 0; i < 9; i = i + 1)
        multData[i] <= kernel[i] * i_pixel_data[i*8+: 8];
    multDataValid <= i_pixel_data_valid;
end

always @(*) begin
    sumDataInt = 0;
    for (i = 0; i < 9; i = i + 1)
        sumDataInt = sumDataInt + multData[i];
end

always @(posedge i_clk) begin
    sumData <= sumDataInt;
    sumDataValid <= multDataValid;
end

always @(posedge i_clk) begin
    if (i_pixel_data_valid) begin
        if (col_counter == 511) begin
            col_counter <= 0;
            if (row_counter == 511)
                row_counter <= 0;
            else
                row_counter <= row_counter + 1;
        end else begin
            col_counter <= col_counter + 1;
        end
    end
end

```

end

```
always @(posedge i_clk) begin
    if (i_rst) begin
        o_convolved_data    <= 0;
        o_convolved_data_valid <= 0;
        left_pixel          <= 0;
    end else if (sumDataValid) begin
        conv_result    <= sumData / 9;
        top_pixel      <= row_buffer[col_counter];
        left_pixel_curr <= left_pixel;

        if ((row_counter % 2 == 0) && (col_counter % 2 == 0))
            o_convolved_data <= (top_pixel + left_pixel_curr) >> 1;
        else
            o_convolved_data <= conv_result;

        o_convolved_data_valid <= 1;
        row_buffer[col_counter] <= conv_result;
        left_pixel <= conv_result;
    end else begin
        o_convolved_data_valid <= 0;
    end
end
end
```

endmodule

#matrix rebuild-condensed version

```
module conv (
    input      i_clk,
    input      i_rst,
    input  [71:0] i_pixel_data,
    input      i_pixel_data_valid,
    output reg [7:0] o_convolved_data,
    output reg      o_convolved_data_valid
);

    reg [7:0] p [0:8]; // 3x3 pixels
    real G[0:26];      // 27-element feature vector
    real F[0:2];        // Output features
    real mean_out;

    // Kernels
    real k1[0:8]; // Edge
    real k2[0:8]; // Sharpen
```

```

real k3[0:8]; // Blur

integer i;

initial begin
    // Edge detection kernel
    k1[0]=-1; k1[1]=-1; k1[2]=-1;
    k1[3]=-1; k1[4]= 8; k1[5]=-1;
    k1[6]=-1; k1[7]=-1; k1[8]=-1;

    // Sharpen kernel
    k2[0]= 0; k2[1]=-1; k2[2]= 0;
    k2[3]=-1; k2[4]= 5; k2[5]=-1;
    k2[6]= 0; k2[7]=-1; k2[8]= 0;

    // Blur kernel (mean)
    for (i = 0; i < 9; i = i + 1)
        k3[i] = 1.0 / 9.0;
end

always @(posedge i_clk) begin
    if (i_rst) begin
        o_convolved_data <= 0;
        o_convolved_data_valid <= 0;
    end else if (i_pixel_data_valid) begin
        // Extract pixels
        for (i = 0; i < 9; i = i + 1)
            p[i] = i_pixel_data[i*8 +: 8];

        // Inheritance simulation
        for (i = 0; i < 9; i = i + 1) begin
            G[i] = p[i]; // 25%
            G[i+9] = (i > 0) ? p[i-1] : p[i]; // 50%
            G[i+18] = (i > 1) ? p[i-2] : p[i]; // 75%
        end

        // Feature fusion
        F[0] = 0; F[1] = 0; F[2] = 0;
        for (i = 0; i < 9; i = i + 1) begin
            F[0] = F[0] + G[i] * k1[i];
            F[1] = F[1] + G[i+9] * k2[i];
            F[2] = F[2] + G[i+18] * k3[i];
        end

        mean_out = (F[0] + F[1] + F[2]) / 3.0;
    end
end

```

```
// Clamp and assign output
if (mean_out > 255)
    o_convolved_data <= 8'd255;
else if (mean_out < 0)
    o_convolved_data <= 8'd0;
else
    o_convolved_data <= mean_out;

    o_convolved_data_valid <= 1'b1;
end else begin
    o_convolved_data_valid <= 1'b0;
end
end
endmodule
```

REPORTS:

1. Convolution : (Blurred)

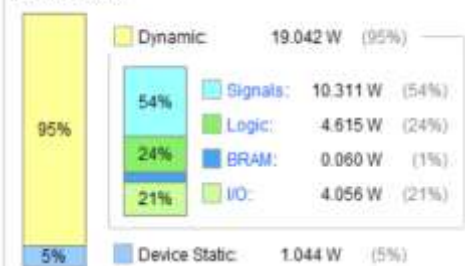
Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): inf	Worst Hold Slack (WHS): inf	Worst Pulse Width Slack (WPWS): NA
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): NA
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: NA
Total Number of Endpoints: 9578	Total Number of Endpoints: 9578	Total Number of Endpoints: NA

All user specified timing constraints are met.

2.

On-Chip Power



Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (28600)	Block RAM Tile (140)	Bonded IOB (200)	BUFGCTRL (32)
ImageProcessTop1	1690	261	96	0.5	23	1
conv (conv)	61	93	0	0	0	0
IC (imageControl)	1585	108	96	0	0	0
IB0 (lineBuffer)	369	18	24	0	0	0
IB1 (lineBuffer_0)	369	18	24	0	0	0
IB2 (lineBuffer_1)	369	18	24	0	0	0
IB3 (lineBuffer_2)	370	18	24	0	0	0
OB (outputBuffer)	43	60	0	0.5	0	0
U0 (outputBuffer_fit_...	43	60	0	0.5	0	0

